Section 5.4: Write-Ahead Logs (WAL) & Platform Processing Speeds

> ***Study Guide*** *| Part 4 of 4 | System Design Interview Prep Series*

Executive Overview

Write-ahead logs (WAL) are the backbone of durable, high-throughput systems. The core promise: **log the intent before the action**. If the system crashes mid-operation, the log lets you replay or roll back to a consistent state. This section covers WAL mechanics, in-memory storage internals (Redis vs Memcached), Kafka's performance architecture, and how to reason about all three in interviews.

Core mental model: WAL turns random writes (scattered data page updates) into sequential writes (append-only log), gaining 10–100× disk throughput at the cost of writing everything twice — once to the log, once to the data file.

Write-Ahead Log (WAL) Core Mechanics

The Transaction Flow

```
Without WAL:
  Write data page → crash → page half-written → inconsistent state ✗

With WAL:
  1. Write log record (sequential append — fast)
  2. fsync log to disk (durability guarantee)
  3. Modify data page in memory
  4. Transaction "committed" once log is durable
  5. Page written to disk later (background, any order)

Crash recovery:
  Read log from last checkpoint → replay any committed records
  Roll back any incomplete records
  → Consistent state guaranteed
```

Why this works: The log is an *ordered, append-only* record of every intention. Even if the actual data pages never made it to disk, the log lets you reconstruct exactly what happened.

Log Sequence Numbers (LSN)

```
LSN: Monotonically increasing 64-bit integer
  Every WAL record has a unique LSN
  Data pages store the LSN of the last WAL record that modified them

Recovery uses LSNs to know:
  "This page has LSN 5000. WAL has records up to LSN 6000.
   Replay records 5001–6000 against this page."

PostgreSQL example:
  SELECT pg_current_wal_lsn();    → 0/A1B2C3D4
  SELECT pg_walfile_name(pg_current_wal_lsn());
  → 000000010000000000000001  (WAL segment file)
```

WAL Buffer Management

```
// Simplified WAL buffer flush logic (PostgreSQL-like)
struct WALBuffer {
    char   data[8192];    // 8KB ring buffer
    int    write_offset;  // current write position
    int    flush_offset;  // last flushed position
    int    fd;            // WAL file descriptor
};

void wal_write(WALBuffer *buf, WALRecord *rec) {
    // Append record to buffer
    memcpy(buf->data + buf->write_offset, rec, rec->length);
    buf->write_offset += rec->length;

    if (rec->is_commit || buf->write_offset > 6553) {
        // Flush on commit or when 80% full
        write(buf->fd, buf->data + buf->flush_offset,
              buf->write_offset - buf->flush_offset);
        fdatasync(buf->fd);    // Durable to disk
        buf->flush_offset = buf->write_offset;
    }
}
```

Key knob: `synchronous_commit` in PostgreSQL. - `ON` (default): `fsync` on every commit. Fully durable. Latency: ~1–10ms per commit. - `OFF` : Returns success before fsync. Latency: <1ms. Risk: up to ~600ms of commits lost on crash. - `LOCAL` : Durable locally, async to replicas.

Checkpoint Mechanics

Without checkpoints, WAL would grow forever (you'd need to replay from day 1 on every crash).

```
Checkpoint process (every 5 min or ~1 GB of WAL):

1. Mark "checkpoint start" in WAL
2. Background writer flushes all dirty data pages to disk
3. Sync all dirty pages: fdatasync on each data file
4. Write "checkpoint complete" to WAL
5. WAL segments before this checkpoint can now be recycled/deleted

Recovery after crash:
  Start from: last checkpoint_complete record
  Replay: WAL records between that LSN and crash point
  Time: O(WAL since last checkpoint), not O(all history)
```

ARIES Protocol (Algorithm for Recovery and Isolation Exploiting Semantics): The formal foundation used by most production DBs.

```
Three phases:
1. Analysis:  Scan WAL from checkpoint to determine dirty pages and active txns
2. REDO:      Re-apply all logged changes (even uncommitted) to reach crash state
3. UNDO:      Roll back uncommitted transactions using undo log records
```

Redis vs Memcached — Architecture Deep Dive

Redis: Single-Threaded Event Loop

Redis uses a single-threaded reactor model. This sounds like a limitation but is a strength for its use case.

```
# Simplified Redis event loop (pseudocode)
while True:
    # epoll: wait for I/O events (up to 10,000 sockets)
    events = epoll_wait(fd_set, timeout=100ms)

    for event in events:
        if event.readable:
            # Read client command
            cmd = parse_redis_protocol(event.client_fd)
            # Execute command — no locks needed (single thread)
            result = execute_command(cmd)
            write_response(event.client_fd, result)
```

```
        if event.writable:
            # Flush pending output buffer
            flush_output_buffer(event.client)
```

Why single-threaded works: Redis operations are in-memory — they take microseconds. The bottleneck is network I/O, not CPU. epoll handles 10K+ connections efficiently without blocking.

Redis 6.0+ change: I/O threads for network reads/writes (multi-threaded), but command *execution* remains single-threaded. This removes the network I/O bottleneck while preserving atomicity.

Redis Internal Data Encodings

Redis automatically chooses encoding based on value size to minimize memory:

```
String → SDS (Simple Dynamic Strings):
  [length | free | data...]
  O(1) strlen (length cached), binary-safe, avoids frequent realloc

List → Listpack (small) or LinkedList (large):
  Listpack: <128 elements, all < 64 bytes → compact memory block
  LinkedList: For larger lists → O(N) traversal, more memory

Sorted Set → Listpack (small) or SkipList + HashTable (large):
  Listpack: <128 members, all scores/members < 64 bytes
  SkipList: O(log N) ordered operations (ZADD, ZRANGE, ZRANGEBYSCORE)
  HashTable: O(1) member→score lookups

  Why both? SkipList alone doesn't give O(1) score lookup by member name.
```

Memcached: Multi-Threaded Slab Allocator

```
Thread pool: 4-8 worker threads (configurable)
  Each thread: subset of connections via libevent
  CAS (Compare-And-Swap) for concurrent updates

Slab allocator — prevents heap fragmentation:
  Slab class 1: items 1-64 bytes
  Slab class 2: items 65-128 bytes
  ...
  Slab class N: items 512KB+

  Memory allocation: O(1) — take from pre-allocated slab
  No fragmentation from mixed-size items in same page
```

Side-by-Side Comparison

Dimension	Redis	Memcached	Choose Redis when…	Choose Memcached when…
Throughput	~100K ops/sec	~200K ops/sec	Ops are complex	Pure GET/SET at max speed
Data types	Rich (List, Set, ZSet, Hash, Stream)	String only	You need ZSet leaderboards, pub/sub, etc.	Simple KV
Persistence	RDB + AOF	None	Can't afford data loss	Ephemeral cache only
Clustering	Redis Cluster (built-in)	Client-side sharding	Operationally simple cluster	Already have sharding layer
Memory efficiency	SDS overhead	Slab overhead	Mixed workloads	Uniform-size items
Atomic operations	Yes (Lua scripts, transactions)	CAS only	Need multi-key atomicity	CAS is enough

---

Kafka — Why It's So Fast

Kafka's performance comes from four architectural decisions, not raw hardware.

1. Sequential I/O Only

```
Random disk writes: ~200 IOPS → ~1.6 MB/s (HDD)
Sequential writes: ~1,000 IOPS → ~500 MB/s (HDD), 3+ GB/s (NVMe)

Kafka always appends to the end of partition log files.
No random seeks. Even HDDs can sustain 500 MB/s for sequential appends.

This is why Kafka's write throughput often exceeds Redis (pure in-memory):
  Redis: ~1 GB/s network-bound
  Kafka: 2+ GB/s sequential disk writes, batched and compressed
```

2. Zero-Copy (sendfile)

```
Traditional file read + send:
  Disk → Kernel page cache → User buffer (copy 1) → Socket buffer (copy 2) → NIC →
Network
  2 copies through userspace

Zero-copy (sendfile syscall):
  Disk → Kernel page cache → NIC → Network
  0 copies through userspace (DMA directly)

Latency: Saves 1–2 memcpy calls per message batch
Throughput: ~3× improvement for large batches
```

```
// Kafka broker internally calls:
ssize_t sent = sendfile(socket_fd, file_fd, &offset, batch_size);
// No user-space buffer involved
```

3. OS Page Cache as Write-Back Buffer

```
Kafka does NOT manage its own memory cache (unlike most databases).
Instead, it relies on the OS page cache:

Write: Producer → Socket → OS page cache → (async) Disk
Read: Consumer → OS page cache (if hot) → Return (no disk read)

Benefits:
  - Pages shared between producer and consumer (JVM vs OS cache)
  - OS handles buffer eviction (LRU)
  - Consumer replaying recent messages = 100% cache hit rate

Risk: Unflushed pages lost on power loss without UPS
Mitigation: acks=all + min.insync.replicas=2 (replica on another machine)
```

4. Batching and Compression

```
Producer batch:
  Buffer messages for up to linger.ms=5ms or batch.size=16KB
  Compress batch (LZ4, Snappy, zstd)
  Send single compressed batch to broker

Effect:
  1M small messages individually: 1M syscalls, 1M headers
  Batched into 1000 batches: 1000 syscalls, 10% of network bytes
  Throughput: 2M msgs/sec achievable with batching
```

Kafka Architecture Summary

```
Topic: logical category
  Partition: ordered, immutable log (one or more per topic)
    Segment: 1 GB log file (rotated when full)
      Record: key, value, timestamp, offset

Partition layout on disk:
/data/topic-name/partition-0/
  00000000000000000000.log      (segment 0, offset 0-999999)
  00000000000000000000.index    (sparse offset index)
  00000000000000001000000.log   (segment 1, active)

Consumer: Reads via offset. No deletion by consumer. Multiple consumers can
  read independently at different offsets (unlike a queue).

ISR (In-Sync Replicas):
  Replicas within 10s of leader → ISR set
  Message "committed" = written to all ISR replicas
  Failover: Any ISR member can become leader
```

Performance Reference Numbers

System	Throughput	p99 Latency	Durability
Redis GET/SET	100K–500K ops/sec	<1ms	Optional (RDB/AOF)
Memcached GET	200K–1M ops/sec	<1ms	None
Kafka produce (batched)	2M msgs/sec	2–5ms same AZ	Replicated
PostgreSQL WAL write	50K–100K TPS	5–20ms	WAL + fsync
NVMe sequential write	3–7 GB/s	<0.1ms	Hardware-level
HDD sequential write	100–250 MB/s	2–10ms	Hardware-level

Interview Question 1 — Exactly-Once Delivery for Financial Transactions

Prompt: How do you use a write-ahead log to implement exactly-once delivery semantics for a financial transaction system?

The Core Problem

```
"At-least-once" is easy: retry on failure → double charges possible
"At-most-once" is easy: no retry → missed transactions possible
"Exactly-once" = both, requiring deduplication and idempotency
```

Solution: WAL + Idempotency Keys

```
def process_payment(tx_id: str, account_id: str, amount: float):
    """
    Exactly-once: Safe to call multiple times with same tx_id.
    """

    # Step 1: Check WAL for existing record (idempotency check)
    existing = wal.query("SELECT status FROM transactions WHERE tx_id = ?", tx_id)
    if existing and existing['status'] == 'completed':
        return {"status": "already_processed", "tx_id": tx_id}

    # Step 2: Log INTENT (before doing anything)
    wal_lsn = wal.append({
        "tx_id": tx_id,
        "type": "debit",
        "account": account_id,
        "amount": amount,
        "status": "pending",
        "timestamp": time.time()
    })
    wal.fsync()   # Durable before proceeding

    # Step 3: Execute (may fail — that's OK, we can replay)
    try:
        result = accounts_db.debit(account_id, amount)

        # Step 4: Log COMPLETION
        wal.update(wal_lsn, {"status": "completed", "result": result})
        wal.fsync()
        return {"status": "success", "tx_id": tx_id}

    except InsufficientFundsError as e:
        wal.update(wal_lsn, {"status": "failed", "reason": "insufficient_funds"})
        raise

    except Exception as e:
        # Don't update WAL — leave as 'pending' for recovery
        raise
```

Recovery on restart:

```
def recover_on_startup():
    pending_txns = wal.query("SELECT * FROM transactions WHERE status =
```

```
'pending'")

    for txn in pending_txns:
        # Was the account actually debited? Check authoritative source.
        if accounts_db.was_debited(txn['tx_id']):
            wal.update(txn['lsn'], {"status": "completed"})
        else:
            # Safe to re-execute (idempotent check prevents double-charge)
            process_payment(txn['tx_id'], txn['account'], txn['amount'])
```

Key insight: The `tx_id` is the idempotency key. If `process_payment` is called twice with the same `tx_id`, the WAL check at Step 1 short-circuits. The WAL provides the persistent record of what happened.

Interview Question 2 — WAL-Based Replication Lag Monitoring

Prompt: Your primary PostgreSQL DB replicates to a read replica. How do you detect and alert on replication lag? What's the maximum safe lag for a read-heavy analytics service?

Solution: LSN-Based Lag Measurement

```
-- On primary: current WAL position
SELECT pg_current_wal_lsn() AS primary_lsn;

-- On replica: last received and applied position
SELECT received_lsn, replay_lsn,
       pg_wal_lsn_diff(pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn()) AS
apply_lag_bytes
FROM pg_stat_replication;

-- Human-readable lag (primary perspective)
SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag;
```

Lag analysis by use case:

```
Analytics dashboards: lag < 30s acceptable (data freshness for trends)
  → Configure: max_standby_streaming_delay = 30s

User-facing read queries (profile, feed): lag < 5s
  → Route "read your own writes" to primary for 5s post-write

Financial reporting: lag = 0 (always read from primary)
  → Never route to replica for regulated data
```

Alert thresholds:

```
LAG < 1s:    Healthy (normal network + CPU variance)
LAG 1–10s:   Warning (long-running query blocking apply)
LAG > 10s:   Critical (network partition or replica overloaded)
LAG > 60s:   Emergency (failover candidate degraded)
```

Interview Question 3 — Designing a Durable Event Queue with WAL Semantics

Prompt: Build a simple durable task queue (like SQS) where: tasks survive server restarts, tasks are processed exactly once, and throughput must be >10K tasks/second.

Solution: Append-Only WAL + Offset-Based Processing

```
Design:
  Storage: WAL-structured append-only file (like Kafka's log segment)
  Producers: Append to WAL (sequential write)
  Consumers: Track "processed up to offset N" in their own state

  Exactly-once: Consumer marks task as processed in its own WAL before returning

WAL record format:
  [offset(8B) | timestamp(8B) | length(4B) | payload(variable)]

Producer path (fast — sequential append only):
  lock.acquire()
  offset = current_offset
  file.write(encode(offset, timestamp, payload))
  file.flush()   # or fdatasync for full durability
  current_offset += 1
  lock.release()
  → Throughput: ~100K writes/sec (NVMe sequential)

Consumer path:
  1. Read record at consumer_checkpoint_offset
  2. Process task (application logic)
  3. Write to consumer's own offset log: "processed offset N"
  4. consumer_checkpoint_offset += 1

  Crash recovery:
  - Consumer restarts: reads its offset log → resume from last checkpoint
  - Re-processes tasks since last checkpoint → idempotent handlers needed
```

Throughput math:

```
WAL append: 10K × avg 1KB payload = 10 MB/s sequential
NVMe: 3 GB/s sequential capacity → 10 MB/s = 0.3% utilization
```

```
Bottleneck: Network (at 10 Gbps: ~1.25 GB/s) — not disk

Batching (like Kafka): Group 100 tasks per fsync
  100 appends → 1 fdatasync: 100× throughput improvement
  100K tasks/sec with single-file WAL on NVMe
```

Trade-off vs. using Kafka directly:

```
Custom WAL queue:
  Pros: Simple, low latency, no ZooKeeper/KRaft dependency
  Cons: No replication, no consumer groups, no compaction

Use Kafka when:
  Multiple consumer groups with different offsets
  Multi-partition parallelism needed
  Long-term message retention with compaction
  Cross-datacenter replication
```

Summary: WAL Mental Model for Interviews

```
Any time you need:
  ✓ Durability on single node → WAL + fsync
  ✓ Crash recovery → WAL replay
  ✓ Exactly-once semantics → WAL + idempotency keys
  ✓ Replication → WAL streaming (logical or physical)
  ✓ Point-in-time recovery → WAL archiving

WAL trade-off:
  Cost: Every write is 2× (log + data page)
  Gain: Sequential I/O (10–100× faster than random), guaranteed consistency

Platform speed rules of thumb:
  L1 cache: 1 ns
  RAM: 100 ns
  NVMe SSD sequential: 0.1 ms
  HDD sequential: 5–10 ms
  Network same-AZ: 0.5 ms
  Network cross-region: 50–100 ms

The entire WAL design philosophy: make the critical path (commit) touch
only sequential disk I/O (fast), and push random I/O to the background.
```